

GitHub Repository Analysis Report

Generated by GitHub Repository Analyzer with AI Assistant

Repository Overview

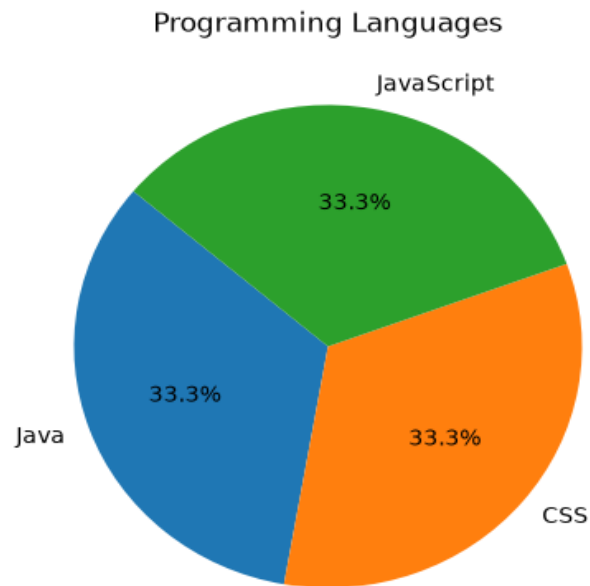
Repository Name	Hotstar-App_f5b1d563
Default Branch	master
Branches	1
Commits	24
Contributors	4

Repository Statistics

Statistic	Value
Total Files	81
Total Directories	20

Programming Languages

Language	Files	Percentage
Java	2	33.3%
CSS	2	33.3%
JavaScript	2	33.3%



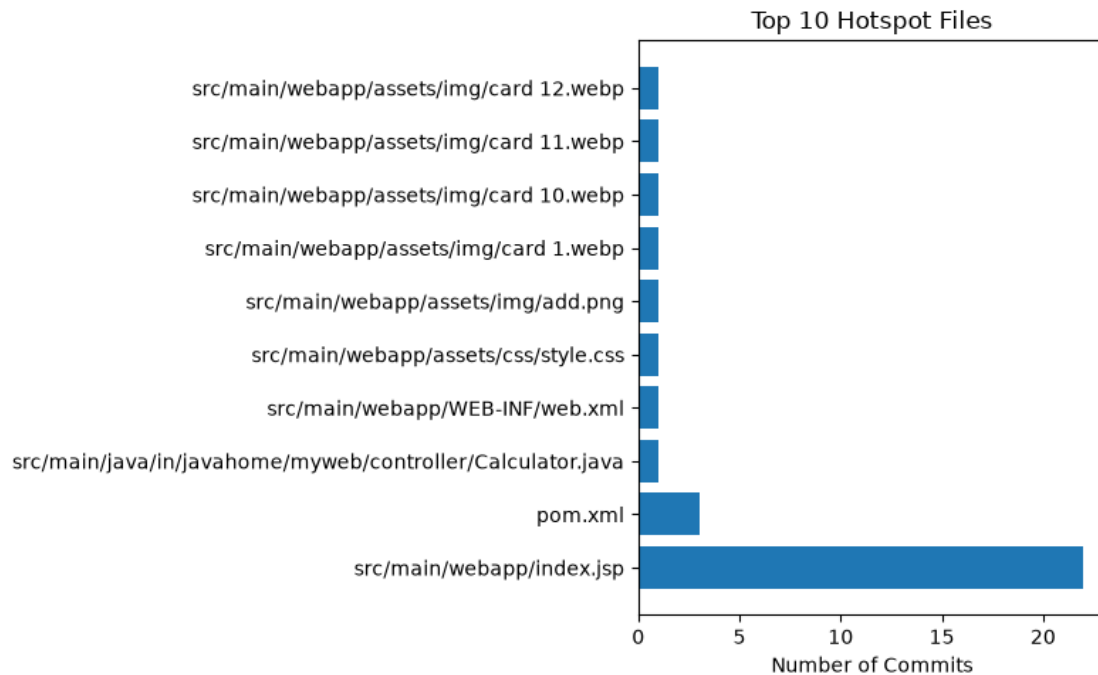
Dependencies

Dependency
No dependencies found

Hotspot Files

File	Commits
src/main/webapp/index.jsp	22
pom.xml	3
src/main/java/in/javahome/myweb/controller/Calculator.java	1
src/main/webapp/WEB-INF/web.xml	1
src/main/webapp/assets/css/style.css	1
src/main/webapp/assets/img/add.png	1
src/main/webapp/assets/img/card 1.webp	1
src/main/webapp/assets/img/card 10.webp	1
src/main/webapp/assets/img/card 11.webp	1

src/main/webapp/assets/img/card 12.webp	1
---	---



AI Repository Summary

Repository Purpose

A web application clone or prototype inspired by the Hotstar streaming platform, built with a Java backend and web front-end technologies.

Technologies Used

- Java
- Maven
- JavaScript
- CSS
- HTML

Project Organization

Structured as a standard Maven application centered around the src directory containing Java backend logic and frontend assets, managed with a pom.xml configuration file.

Intended Users

Software developers, students, and learners interested in Java web development and video streaming UI clones.

Beginner Summary

This project is a replica or educational demo of the Hotstar streaming service, demonstrating how to build a basic web application using Java, JavaScript, and CSS.

AI Recommendations

Repository Strengths

- ✓ Adheres to standard Maven project directory structure with a root pom.xml and src layout
- ✓ Includes full-stack components combining Java backend services with web frontend assets (CSS, JavaScript)
- ✓ Structured directory organization with 20 directories indicating modular separation of concerns

Areas for Improvement

- Complete lack of documentation; missing README.md with setup instructions, architecture overview, and requirements
- No automated CI/CD pipeline configurations or containerization files (e.g., Dockerfile, GitHub Actions) detected
- Missing test configuration or explicit verification pipelines visible in root configurations

Best Practices

- Add a comprehensive README.md detailing project purpose, prerequisites, build steps, and environment configuration
- Implement continuous integration workflows to automate build, testing, and static code analysis
- Ensure standard .gitignore files are present to avoid committing build artifacts and IDE metadata

Overall Repository Quality

Needs Improvement